

Persistence — Storage & RocksDB

Where the ledger lives on disk — an embedded key-value store under a vertex-aware abstraction, and why RocksDB rather than MongoDB or SQL.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 27 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

RocksDB · Embedded KV store · LSM-tree · Column families · Key design ·
TransactionStorage · Backends · DAG traversal · Allow-scope · Crash safety

Table of Contents

Chapter 27 — Persistence: Storage & RocksDB	3
27.1 Localization	3
27.2 What it does and why it exists	5
27.3 The concepts it rests on	5
The new idea: an embedded key-value store	6
27.4 The code, walked — a tiny generic store first	8
27.5 The byte layer: RocksDBStorage	8
27.6 The vertex layer: TransactionStorage and its backends	10
Saving a vertex	11
Reading a vertex back	12
Key design and column families, summarized	13
The narrow read interface: VertexStorageProtocol	14
27.7 Walking the DAG, and the allow-scope	14
Traversal helpers	14
Allow-scope: gating by validation state	15
27.8 Crash safety — the flag in the database	16
27.9 The trade-off — why RocksDB and not the alternatives	17
27.10 How it plugs into the lifecycle	18
Recap	19

Chapter 27 — Persistence: Storage & RocksDB

What you'll learn in this chapter

- What an **embedded key-value store** is, and what makes **RocksDB** one — the LSM-tree, column families, and ordered keys — at a level you can reason about without reading C++.
- How `hathor-core` layers a low-level byte store (`hathor/storage/`) under a **vertex-aware** store (`hathor/transaction/storage/`), and why that two-layer split exists.
- How a single vertex is written and read back: which bytes go where (the body, the mutable metadata, the static metadata — three slots), and how the bytes become a live `Block` or `Transaction` again.
- How the storage **walks the DAG** (BFS/DFS helpers), what an **allow-scope** is, and how a stored flag lets the node detect that it crashed last time.
- The decision itself: **why RocksDB and not MongoDB, not a SQL database, not plain files** — with the trade-offs stated honestly.

By now you have met the data and its shape. Chapter 25 defined the **vertex model** in code — `BaseTransaction` and its subclasses `Block` and `Transaction`, their inputs and outputs, and the mutable per-vertex **metadata**. Chapter 26 showed how a vertex is turned into a compact stream of bytes and back (the bespoke serialization). This chapter is the next link in that chain: once you have the bytes, **where do they go, and how do you get them back fast?** That is persistence, and in `hathor-core` it rests on an embedded database called RocksDB.

This is the book's canonical treatment of RocksDB. Earlier chapters dropped a one-line footnote ("an embedded key-value store"); here we open it up.

27.1 Localization

Persistence lives in **two** packages, deliberately stacked. The lower one knows nothing about Hathor; it speaks raw bytes. The upper one knows everything about vertices.

```

hathor-core/
└─ hathor/
    └─ storage/
        ├── __init__.py
        └─ rocksdb_storage.py
    └─ transaction/
        └─ storage/
            ├── __init__.py
            ├── transaction_storage.py
            ├── rocksdb_storage.py
            ├── memory_storage.py
            ├── vertex_storage_protocol.py
            ├── traversal.py
            ├── tx_allow_scope.py
            ├── exceptions.py
            └─ migrations/

```

- ← LOW LEVEL: raw byte key-value store (exports RocksDBStorage)
- ← RocksDBStorage: open the DB, hand out column families, put/get bytes ◀ YOU ARE HERE
- ← HIGH LEVEL: vertex-aware store (exports TransactionStorage, ...)
- ← TransactionStorage (ABC, :92) + BaseTransactionStorage (shared logic, :1013)
- ← TransactionRocksDBStorage (the production backend)
- ← (now empty – see §27.6)
- ← VertexStorageProtocol (a narrow read interface)
- ← BFS / DFS walks over the DAG
- ← TxAllowScope: VALID / PARTIAL / INVALID gating
- ← TransactionDoesNotExist, ...
- ← on-disk schema migrations

The naming collision is worth flagging once so it never trips you: there are **two** files called `rocksdb_storage.py`. `hathor/storage/rocksdb_storage.py` is the byte layer (class `RocksDBStorage`). `hathor/transaction/storage/rocksdb_storage.py` is the vertex layer (class `TransactionRocksDBStorage`). The first does not import or know about transactions; the second owns an instance of the first.

Recap — the module map (full treatment in Ch. 0 §0.4). Chapter 0 grouped the packages by the five jobs of a full node. The very first job was store the ledger: " `storage/` is a thin wrapper around RocksDB ... `transaction/storage/` sits on top and speaks in vertices." This chapter is the zoom-in on that one sentence. The derived lookups — "what does this address own?" — are a separate job, handled by `indexes/` in Chapter 28.

Context. Everything else in the node is downstream of storage. Verification reads parents from it; consensus reads and writes metadata to it; the sync agent fills it; the indexes are rebuilt from it; the API serves answers out of it. If storage is wrong, the whole node is wrong — which is exactly why the layer also carries the crash-safety flag (§27.8). It is the one place where "the ledger" stops being an idea and becomes files on a disk.

27.2 What it does and why it exists

Strip away the vocabulary and the storage layer answers four blunt questions:

1. **Where do I put a vertex so it survives a restart?** A node may run for months and then be killed, rebooted, or crash. Everything it learned — every block and transaction it validated — must still be there when it comes back. RAM is not enough; the data must hit a disk.
2. **How do I find one vertex again, fast, out of millions?** The node is constantly asked "do you have the vertex with hash `0xabc...`?" and "give me its parents." With tens of millions of vertices, scanning is out of the question. Lookup by hash must be effectively instant.
3. **How do I store the bookkeeping that changes over time?** A vertex's own contents (its inputs, outputs, parents) never change after creation — they are fixed by the hash. But the node's opinion about that vertex — its accumulated weight, whether it has been voided, its height — changes as the ledger grows. That mutable part is the **metadata**, and it must be stored and updated separately from the immutable body. (Chapter 25 split the metadata in two: a static part computed once at creation and a mutable part rewritten as the ledger evolves; storage keeps both, in their own places — §27.6.)
4. **How do I walk the graph?** Many questions ("what confirms this transaction?", "trace funds back to their source") are answered by following edges through the DAG. The storage layer must let callers traverse from a vertex to its parents, children, and the vertices that spend it.

A full node cannot function without all four. The storage package exists to provide them, and to do so with one more property the others depend on: **integrity**. A half-written record after a power cut must not silently corrupt the ledger. That requirement is most of why the node uses a real database engine instead of writing files by hand.

27.3 The concepts it rests on

Four ideas from earlier chapters carry directly into this one. We recap them briefly, then build the one genuinely new concept — what RocksDB actually is.

Recap — abstraction & interchangeable backends (full treatment in Ch. 1 & Ch. 5). An **abstract base class**¹ declares what operations exist (`save_transaction` , `get_transaction`) without committing to how. Concrete subclasses fill in the how. Because callers depend only on the abstract type, you can swap one implementation for another and nothing upstream notices. Hathor uses this directly: `TransactionStorage` is the abstract contract; the production code stores to RocksDB while tests can use a lighter, in-process variant — and the manager, verification, and consensus are written against the abstract type, never against RocksDB by name.

Recap — the vertex model (full treatment in Ch. 25). A `vertex` is any node of Hathor's ledger graph — a `Block` or a `Transaction` , both subclasses of `BaseTransaction` . Each vertex has an **immutable body** (version, inputs, outputs, parents, nonce) fixed at creation, and a **mutable metadata** object (`TransactionMetadata`) the node maintains as the ledger evolves — height, accumulated weight, `voided_by` , validation state. Hold this split: storage persists the two parts in two different places.

Recap — serialization (full treatment in Ch. 26). Serialization turns a live object into a flat sequence of bytes; deserialization reverses it. Hathor uses a custom compact binary format for vertex bodies. A database stores bytes, so serialization is the step that happens just before a write and just after a read. In this chapter you will see exactly where: `bytes(tx)` for the body on the way in, `VertexParser.deserialize(...)` on the way out, and `TransactionMetadata.to_bytes()` / `.from_bytes()` for the metadata.

Recap — DAG traversal (full treatment in Ch. 8). The ledger is a directed acyclic graph. Two standard ways to visit every reachable node are **breadth-first search** (BFS — explore neighbours level by level, using a queue) and **depth-first search** (DFS — plunge down one path before backtracking, using a stack). Hathor's storage ships both, plus a BFS variant that visits in timestamp order. We see them in §27.7.

The new idea: an embedded key-value store

Two words need unpacking before RocksDB makes sense: **embedded** and **key-value**.

Key-value store. The simplest possible database. You hand it a key (some bytes) and a value (some bytes), and it remembers the pairing. Later you hand it the same key and it returns the value. That is the entire interface: `put(key, value)` , `get(key)` , `delete(key)` . There are no tables, no columns, no `SELECT ... WHERE` , no joins. If a Python `dict` could survive a reboot and hold more data than fits in RAM, it would be a key-value store. The mental model is exactly that: **a dictionary that lives on disk**.

Embedded. The database runs inside your own program's process, as a library you import — not as a separate server you connect to over a network. When `hathor-core` calls `put`, it is a function call into linked C++ code, not a request sent to another machine (or even another process). Contrast a server database like PostgreSQL or MongoDB: there, the database is its own long-running program; your code opens a socket to it and speaks a protocol. Embedded means no socket, no protocol, no second program to install, start, secure, or keep alive. SQLite is the famous embedded SQL database; RocksDB is an embedded key-value one.

RocksDB, then, is an embedded, ordered key-value store written in C++, originally built at Facebook from Google's LevelDB. "Ordered" is the one extra property beyond the dictionary model, and it earns its keep below. Three of its mechanisms matter to us.

The LSM-tree. Most of the reason a database is hard to write from scratch is making writes both fast and durable. A naïve approach — open the file, seek to the right spot, overwrite a few bytes — is slow, because disks (especially spinning ones) hate random seeks, and dangerous, because a crash mid-write leaves a torn record. RocksDB uses a **log-structured merge-tree**². The idea: never overwrite in place. New writes are appended to an in-memory table (and a write-ahead log on disk for durability). When that table fills, it is flushed to disk as a new, immutable, sorted file. Reads check the newest files first. Periodically a background process called **compaction**³ merges the sorted files together, discarding superseded values. The payoff: writes are sequential appends (fast and crash-safe), and reads stay fast because the files are sorted and can be binary-searched. The cost: a value may live in several files until compaction tidies up, so reads occasionally do more work, and compaction consumes background CPU and disk. For a node that writes constantly (every new vertex, every metadata update) this trade is favourable.

Column families. A single RocksDB database can be sliced into named sub-stores called **column families**⁴. Think of them as separate dictionaries that share one database file, one transaction log, and one open handle — but keep their own key-spaces and can be tuned and compacted independently. They are how Hathor avoids key collisions and keeps unlike data apart: the vertex bodies live in one column family, their metadata in another, even though both are keyed by the same 32-byte hash. Without column families you would have to smear a prefix onto every key to keep the two namespaces from clashing.

Ordered keys. Unlike a hash map, a RocksDB column family keeps its keys in sorted byte order, and lets you iterate them in that order (and seek to a position). For lookup-by-hash this ordering is irrelevant — a hash is random, so its sort order is meaningless. But it is what makes "give me every vertex" (a full scan, used at startup to rebuild indexes) a clean sequential read, and it is the property the indexes of Chapter 28 exploit heavily.

27.4 The code, walked — a tiny generic store first

Before the real classes, here is the whole idea in miniature: a persistent dictionary with a put/get interface and a serialization step on each side. No Hathor, no RocksDB — just the shape.

```
import json

class TinyStore:
    """A dict-on-disk: keys are bytes, values are bytes."""
    def __init__(self):
        self._db = {}  # pretend this survives a reboot

    def put(self, key: bytes, value: bytes) -> None:
        self._db[key] = value

    def get(self, key: bytes) -> bytes | None:
        return self._db.get(key)

class AccountStore:
    """A 'vertex-aware' layer: speaks Accounts, not bytes."""
    def __init__(self, backend: TinyStore):
        self._backend = backend

    def save(self, account) -> None:
        key = account.id.encode()
        value = json.dumps(account.to_dict()).encode()  # serialize
        self._backend.put(key, value)

    def load(self, account_id: str):
        value = self._backend.get(account_id.encode())
        if value is None:
            raise KeyError(account_id)
        return Account.from_dict(json.loads(value))  # deserialize
```

Every concept of the real layer is already here. `TinyStore` is the byte layer: it knows keys and values, nothing else. `AccountStore` is the domain layer: it knows how to turn an `Account` into bytes (`to_dict` → `json.dumps`) and back (`json.loads` → `from_dict`), and it picks the key (`account.id`). Swap `TinyStore` for a `RedisStore` or a `RocksDBStore` and `AccountStore` does not change a line — that is the interchangeable-backend pattern. Hold this picture; the production code is the same skeleton with more muscle.

27.5 The byte layer: `RocksDBStorage`

The low layer is small — about a hundred lines in `hathor/storage/rocksdb_storage.py`. Its whole job is: open one RocksDB database and hand out column families.


```
# hathor/storage/rocksdb_storage.py
class RocksDBStorage:
    """ Creates a RocksDB database
        Give clients the option to create column families
    """
    def __init__(self, path, cache_capacity=None) -> None:
        self.log = logger.new()
        # keep a reference to the TemporaryDirectory; it is cleaned up when GC'd
        self.path, self.temp_dir = self._get_path_and_temp_dir(path)
        db_path = os.path.join(self.path, _DB_NAME) # _DB_NAME = 'data_v2.db'
        ...
        self._db = rocksdb.DB(db_path, options, column_families=column_families)
```

A few details deserve a junior-level gloss.

Opening the database (`rocksdb_storage.py:69`). The constructor builds an `Options` object (`rocksdb_storage.py:44`) and opens the database at `<path>/data_v2.db`. The options it sets are operational tuning, not correctness: an 80 MB write buffer instead of the 4 MB default (`rocksdb_storage.py:46`), memory-mapped reads and writes (`:48 - :49`), **no compression** (`:47` — Hathor trades disk space for CPU), and a 3 GB cap on the write-ahead log so the `.log` files cannot grow without bound (`:53`). You do not need to memorize these; the point is that the byte layer is also where disk-engine policy is decided, once, for the whole node.

Discovering existing column families (`rocksdb_storage.py:57 - :67`). RocksDB will not open a database without being told the names of every column family it already contains. So the code first lists them (`list_column_families`, `:59`); if that throws — meaning the database does not exist yet — it calls `repair_db` to create a fresh one and starts with an empty list (`:60 - :63`). Then it opens all of them.

Creating a column family on demand (`rocksdb_storage.py:93`):

```
def get_or_create_column_family(self, cf_name: bytes) -> 'rocksdb.ColumnFamilyHandle':
    cf = self._db.get_column_family(cf_name)
    if cf is None:
        cf = self._db.create_column_family(cf_name, rocksdb.ColumnFamilyOptions())
    return cf
```

This is the seam between the two layers. The vertex layer does not open the database; it asks the byte layer for the named column families it wants, creating them the first time. `get_db()` (`:90`) hands back the raw handle so the upper layer can issue the actual `put / get / delete` calls.

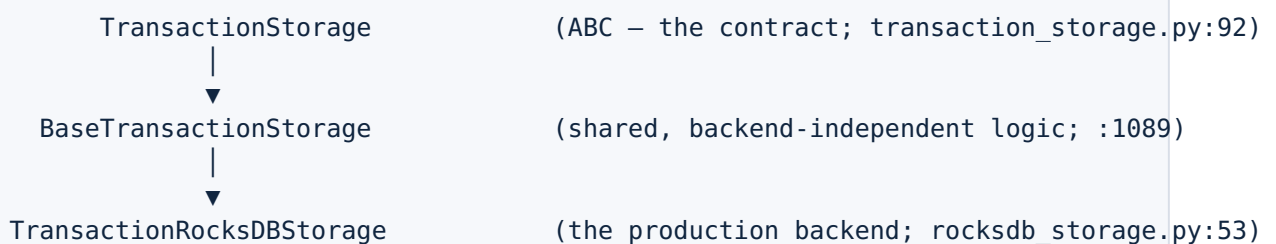
The temp path. `create_temp()` (`rocksdb_storage.py:73`) builds a `RocksDBStorage` backed by a `tempfile.TemporaryDirectory`. The comment at `:38` explains the one subtlety: the object must keep a reference to that `TemporaryDirectory`, because Python deletes the directory when the object is garbage-collected. This is what powers a real-

but-throwaway database — used in tests and by the `--temp-data` CLI flag (Chapter 24) — without spinning up anything different from production. It is the production code path, pointed at scratch space.

That is the entire byte layer. It speaks `(column_family, key_bytes) → value_bytes` and has no idea what a transaction is.

27.6 The vertex layer: `TransactionStorage` and its backends

On top of the byte layer sits the vertex-aware store. Here the class structure matters, so let us name it precisely.



`TransactionStorage` (`transaction_storage.py:92`) is the **abstract base class** — the contract every backend must honour: `save_transaction` (`:425`), `get_transaction` (`:541`), `transaction_exists` (`:490`), `_get_transaction` (`:510`), traversal, allow-scope, and many more. Its own docstring is a candid bit of legacy: "Legacy sync interface, please copy `@deprecated` decorator when implementing methods" (`:93`).

`BaseTransactionStorage` (`transaction_storage.py:1089`, declared `class BaseTransactionStorage(TransactionStorage)`) sits between the contract and the backend. It implements the parts that are the same regardless of where bytes land — the allow-scope validation (`pre_save_validation:442`, `post_get_validation:456`, both calling `_validate_transaction_in_scope:474`), the weak-reference identity map, the genesis handling (`_save_or_verify_genesis:333`), and the index bookkeeping — and leaves the genuinely storage-specific holes (`_get_transaction`, `transaction_exists`, `_save_static_metadata`, `save_transaction`, the migration accessors) as `abstractmethod`s for the backend to fill.

`TransactionRocksDBStorage` (`rocksdb_storage.py:53`) is the concrete backend that fills those holes against RocksDB. In production this is always the one in use.

One detail worth fixing now, because it shapes the read/write code below: the storage keeps an in-memory **identity map** — a `WeakValueDictionary` keyed by hash (`transaction_storage.py:138`). Its purpose is correctness, not speed: it guarantees that at any moment there is at most one live Python object for a given vertex (`_save_to_weakref:352` even asserts this), so two parts of the node that both "load" the

same transaction get the same object and see each other's metadata updates. (A weak reference lets the object be garbage-collected once nobody else holds it.) Separately, the RocksDB backend keeps a real **write-back cache** for throughput, covered next.

About the "memory backend." The book's chapter brief — and the orientation in Chapter 0 — describe an in-memory storage used in tests as the classic second backend proving the abstraction. That pattern is real and the abstraction is genuine, but note the current state of the tree: `hathor/transaction/storage/memory_storage.py` exists but is **empty (0 bytes)**, and the package's `__init__.py` exports only `TransactionStorage`, `TransactionRocksDBStorage`, and `VertexStorageProtocol`. So on this branch the interchangeable-backend design is expressed mainly through (a) the `TransactionStorage` ABC plus `BaseTransactionStorage`, and (b) the throwaway RocksDB created by `create_temp()` / `--temp-data`, which gives tests a real backend on scratch disk rather than a separate in-RAM class. The seam to add another backend is still there — implement the abstract methods — but a populated memory backend is not present here. (Stated plainly because the brief expected one; the code is the authority.)

Saving a vertex

The public `save_transaction` is abstract on the base class, but the base supplies the shared front-half — `pre_save_validation` (`transaction_storage.py:442`, which runs the allow-scope check `_validate_transaction_in_scope:474`) and the static-metadata write (`_save_static_metadata:435`). The RocksDB subclass calls that base method, then writes the body and mutable metadata:

```
# hathor/transaction/storage/rocksdb_storage.py (TransactionRocksDBStorage)
def save_transaction(self, tx, *, only_metadata=False) -> None:           # :202
    super().save_transaction(tx, only_metadata=only_metadata)           # :203 base checks + st
    self._save_transaction(tx, only_metadata=only_metadata)             # :204 into the write-b
    self._save_to_weakref(tx)                                           # :205 into the identit

def _save_transaction(self, tx, *, only_metadata=False) -> None:        # :207
    self._update_cache(tx)                                              # keep the live object
    self.cache_data.dirty_txs.add(tx.hash)                             # mark it "needs writin

def _save_transaction_to_db(self, tx) -> None:                          # :211 (runs on flush)
    key = tx.hash
    self._db.put((self._cf_tx, key), self._tx_to_bytes(tx))           # :214 body → cf_tx (bi
    meta = tx.get_metadata(use_storage=False).to_bytes()               # :215 mutable metadata
    self._db.put((self._cf_meta, key), meta)                           # :216 metadata → cf_met

def _save_static_metadata(self, tx) -> None:                            # :218 (called by the
    self._db.put((self._cf_static_meta, tx.hash), tx.static_metadata.json_dumpb()) # :220 stati
```

This is the heart of the chapter, so trace it slowly.

- **The key is the hash.** The body, the mutable metadata, and the static metadata are all stored under `tx.hash` — the 32-byte identifier of the vertex (Chapter 25) — each in its own column family. The hash is the primary key; there is no separate id, no auto-increment, nothing to look up first. This is the

access pattern that makes a key-value store the right tool: the node almost always already knows the hash of what it wants.

- **Three values, three slots.** The immutable body is encoded with `bytes(tx)` (via `_tx_to_bytes`, :178 / :213) — Hathor's compact binary serialization from Chapter 26 — into `_cf_tx`. The mutable metadata is encoded with `TransactionMetadata.to_bytes()` (:215) into `_cf_meta`. The static metadata is encoded with `static_metadata.json_dumpb()` (:220) into `_cf_static_meta`. The three are kept apart on purpose, mirroring Chapter 25's split: the body is fixed forever, the static metadata is computed once at creation and never changes, and the mutable metadata is rewritten constantly by consensus.
- **only_metadata.** Consensus updates metadata constantly (a new vertex changes the accumulated weight and voided status of old ones) without touching their bodies. The `only_metadata=True` flag is threaded through so a metadata-only update can avoid re-encoding the body. Since the body never changes after creation, rewriting it would be wasted work.

The write-back cache. Notice that `save_transaction` does not immediately hit the disk. `_save_transaction` (:207) only puts the vertex in an in-memory LRU cache (`cache_data.cache`, an `OrderedDict`, configured by `CacheConfig` at `transaction_storage.py:75`) and adds its hash to a `dirty_txs` set. The real disk write (`_save_transaction_to_db`, :211) happens later, in a background flush: a timer (`_start_flush_thread`, :111) periodically uses Twisted's `deferToThread` (Chapter 16) to write all dirty vertices off the reactor thread (`_flush_to_storage`, :127); an evicted-but-dirty vertex is also flushed immediately on eviction (`_cache_popitem`, :138). This is a classic write-back cache: batch writes for throughput, but never lose an update. (It is also why `is_empty` (:305) and `_get_all_transactions` (:287) call `_flush_to_storage` first — they must see the latest writes.)

Reading a vertex back

The base class's `get_transaction` (`transaction_storage.py:541`) applies the allow-scope check (`post_get_validation:456`), then delegates to the abstract `_get_transaction`, which the backend implements with a three-tier lookup:

```

# hathor/transaction/storage/rocksdb_storage.py:241 (TransactionRocksDBStorage)
def _get_transaction(self, hash_bytes) -> 'BaseTransaction':
    if tx := self.cache_data.cache.get(hash_bytes): # :242 1) write-back cache?
        ...; return tx
    if tx := self.get_transaction_from_weakref(hash_bytes): # :248 2) live identity map?
        ...; return tx
    tx = self.get_transaction_from_db(hash_bytes) # :253 3) finally, the disk
    if not tx:
        raise TransactionDoesNotExist(hash_bytes.hex()) # :255
    ...; return tx

def _get_transaction_from_db(self, hash_bytes) -> Optional['BaseTransaction']: # :266
    tx_data = self._db.get((self._cf_tx, hash_bytes)) # :268 fetch body bytes
    meta_data = self._db.get((self._cf_meta, hash_bytes)) # :269 fetch metadata bytes
    if tx_data is None:
        return None # :271
    tx = self._load_from_bytes(tx_data, meta_data) # :273 reassemble body + mutable meta
    self._load_static_metadata(tx) # :274 attach static meta (cf_static_...)
    return tx

def _load_from_bytes(self, tx_data, meta_data) -> 'BaseTransaction': # :170
    tx = self.vertex_parser.deserialize(tx_data) # :173 bytes → vertex
    tx._metadata = TransactionMetadata.from_bytes(meta_data) # :174 bytes → mutable met
    tx.storage = self # :175 back-pointer
    return tx

```

Reading is the mirror of writing, and it is where Chapter 26 pays off. A lookup checks the **write-back cache** first (:242), then the **weakref identity map** (:248 — so a vertex already live elsewhere is reused, not duplicated), and only then the **disk** (:253). On a disk read, the body bytes go through `vertex_parser.deserialize(...)` (:173) — the `VertexParser` reads the version byte, dispatches to the right class, and rebuilds a live `Block` or `Transaction` (Chapter 25/26). The mutable metadata bytes are parsed with `TransactionMetadata.from_bytes(...)` (:174); the static metadata is loaded separately from its own column family by `_load_static_metadata` (:222 , using `VertexStaticMetadata.from_bytes`). Finally the reconstructed vertex is given a back-pointer to the storage (`tx.storage = self` , :175) so that it can later fetch its own parents and children on demand — which is how traversal works without the walker holding the whole graph in memory.

If the body key is missing, `_get_transaction` raises `TransactionDoesNotExist` (:255). `transaction_exists` (:232) answers the cheaper question directly: it checks the cache, then RocksDB's `key_may_exist` bloom-filter probe ⁵ , then confirms with a real `get` .

Key design and column families, summarized

one RocksDB database (data_v2.db)			
└─ cf b'tx'	key = vertex hash (32 B)	value = bytes(tx)	[binary b
└─ cf b'meta'	key = vertex hash (32 B)	value = TransactionMetadata	[mutable
└─ cf b'static-meta'	key = vertex hash (32 B)	value = VertexStaticMetadata	[fixed-at
└─ cf b'attr'	key = attribute name	value = small string	[node-lev
└─ cf b'migrations'	key = migration name	value = state byte	[schema v

There are **five** column families; their names are constants at `rocksdb_storage.py:46 - :50` (`_CF_NAME_TX`, `_CF_NAME_META`, `_CF_NAME_STATIC_META`, `_CF_NAME_ATTR`, `_CF_NAME_MIGRATIONS`), resolved to handles in the constructor (`:72 - :76`). The split of metadata into two families — `meta` (mutable) and `static-meta` (immutable) — is the on-disk expression of Chapter 25's static-vs-mutable metadata distinction. The `attr` family is a small general-purpose key→string store the upper layer exposes through `add_value` / `get_value` / `remove_value` (`rocksdb_storage.py:338 / :344 / :341`) — this is where the node-level flags of §27.8 live. The `migrations` family records, per named migration, whether it has run (`get_migration_state` / `set_migration_state`, `:181 / :188`), so an upgraded node can fix up an old on-disk layout once and remember it did — the migration runner itself lives on the base class (`_check_and_apply_migrations`, `transaction_storage.py:214`).

The narrow read interface: `VertexStorageProtocol`

Not every caller needs the full storage. Verification, for instance, only ever reads a handful of things. `vertex_storage_protocol.py:22` defines a **Protocol**⁶ — a structural interface — exposing just four read methods: `get_vertex`, `get_block`, `get_parent_block`, `get_best_block_hash`. Its docstring (`:23`) explains the intent: a verification method can accept "a RocksDB storage or an ephemeral simple memory storage" — anything shaped like these four methods. This is the minimal-surface idea from Chapter 5: depend on the smallest interface that does the job, so more kinds of object can satisfy it.

27.7 Walking the DAG, and the allow-scope

Traversal helpers

`traversal.py` provides three ready-made walkers over the graph, all subclasses of one `GenericWalk` base (`traversal.py:54`): `BFSTimestampWalk` (`:182`, BFS ordered by timestamp via a heap), `BFSOrderWalk` (`:218`, plain BFS via a queue), and `DFSWalk` (`:249`, DFS via a stack). The three differ only in their to-visit container — a heap, a deque, or a list — which is the textbook distinction between BFS and DFS made concrete: **same algorithm, different bag for the pending work**.

Two design choices make these walkers usable across the whole node.

They walk over a `VertexStorageProtocol`, not a concrete store (`traversal.py:61`). A walker fetches each neighbour with `self.storage.get_vertex(_hash)` (`:129`). Because it depends only on the narrow protocol, the same traversal code runs over the real RocksDB store or any stand-in.

They distinguish the two kinds of edge Chapter 8 insisted on. A vertex has verification edges (parents/children — the confirmation DAG) and funds edges (inputs/spent-outputs — the money DAG). The walker's `is_dag_verifications` and `is_dag_funds` flags (`traversal.py:107`, `:113`) pick which edges to follow, and `is_left_to_right` picks the direction (toward the tips, or back toward genesis). So one mechanism answers both "what confirms this?" and "where did these funds come from?".

The control flow is cooperative: `run()` (`:148`) yields each vertex to the caller, and the caller decides per-vertex whether to keep going via `add_neighbors()` or stop via `skip_neighbors()` (`:136`, `:142`). That lets a caller prune branches it does not care about instead of walking the entire reachable graph.

Allow-scope: gating by validation state

Recap — validation state (full treatment in Ch. 25). Every vertex carries a validation state in its metadata. The three that matter here: **VALID** (fully verified and connected), **PARTIAL** (known but not yet fully verified — common mid-sync), and **INVALID**. Sync can store a vertex before all its dependencies have arrived, so the storage must sometimes hold not-yet-valid data.

That creates a hazard: most of the node should only ever see fully-valid vertices, but during startup and sync the storage legitimately contains partial ones. The **allow-scope** is the guard. `TxAllowScope` (`tx_allow_scope.py:25`) is a bit-flag enum — `VALID`, `PARTIAL`, `INVALID`, and `ALL` (`:31 - :34`) — and the storage holds a "current scope," defaulting to `VALID` only (`transaction_storage.py:152`). The enum's `is_allowed(tx)` method (`tx_allow_scope.py:36`) checks a vertex's validation state against the scope; the storage's `save_transaction` and `get_transaction` both run it (via `_validate_transaction_in_scope`, `transaction_storage.py:474`), and a vertex outside the scope is treated as if it does not exist — the storage raises `TransactionNotInAllowedScopeError` (`exceptions.py:50`), a subclass of `TransactionDoesNotExist`.

The scope is switched with a context manager so it always reverts:

```
# hathor/transaction/storage/tx_allow_scope.py:52
@contextmanager
def tx_allow_context(tx_storage, *, allow_scope):
    previous_allow_scope = tx_storage.get_allow_scope()
    try:
        tx_storage.set_allow_scope(allow_scope)
        yield
    finally:
        tx_storage.set_allow_scope(previous_allow_scope)
```

This is the "scope" Chapter 0 §0.3 mentioned in the startup story. You can see it explicitly in the manager: just before initializing components it widens the scope (`manager.py:315`, `set_allow_scope(VALID | PARTIAL | INVALID)`) so it can load and connect partial data, and immediately after it narrows back to valid-only (`manager.py:317`). The context-manager form guarantees the relaxed window always closes, because `finally` restores the previous value even on error. The base class also exposes convenience wrappers — `allow_partially_validated_context()` / `allow_invalid_context()` (`transaction_storage.py:386` / `:394`) and the query `is_only_valid_allowed()` (`:402`) — so callers can widen the scope locally or ask which regime they are in.

27.8 Crash safety — the flag in the database

A full node must not act on a half-built database. If the previous run died mid-write — power cut, `kill -9`, an unhandled exception in consensus — the on-disk metadata may be inconsistent, and trusting it could corrupt the ledger going forward. Hathor guards against this with a flag stored in the database itself.

The mechanism is the small `attr` column family from §27.6. Two attribute keys carry the state, declared on the abstract base: `_manager_running_attribute = 'manager_running'` (`transaction_storage.py:106`) and `_full_node_crashed_attribute = 'full_node_crashed'` (`:109`). The storage sets, clears, and reads these flags through the generic attribute interface (`add_value` / `get_value` / `remove_value`, `rocksdb_storage.py:338` / `:344` / `:341`).

The manager wires them into startup. The first lines of `HathorManager.start` check both flags before doing anything else:

```
# hathor/manager.py:285
if self.tx_storage.is_full_node_crashed():
    self.log.error("... it wasn't stopped correctly. The storage is not reliable anymore ...")
    sys.exit(-1)                                     # :291 refuse to start
if self.tx_storage.is_running_manager():             # :296 still marked "running"?
    self.log.error("... it wasn't stopped correctly ...")
    sys.exit(-1)                                     # :302 refuse to start
...
self.tx_storage.start_running_manager(self._execution_manager) # :343 set the "running" flag
```

If a previous run set the "running" flag and never cleared it (because `stop` never ran — a crash), the next start sees it still set and exits, telling the operator to discard the storage and re-sync. The comment at `manager.py:293 - :295` spells out the subtlety: only the metadata may be wrong after a crash, not the blocks and transactions themselves — but that is enough to make the bookkeeping untrustworthy. The split is deliberate: the policy ("crashed → refuse to start") lives in the manager (Chapter 29), but the durable bit it reads lives in storage, because storage is the only thing that survives a crash.

The intuition is the same as a journaling filesystem's "dirty" flag, or the lock file a long-running program leaves behind: a single persisted boolean that says "I did not finish cleanly last time, so check yourself before you proceed." It costs one tiny key and saves the node from compounding a crash into corruption.

27.9 The trade-off — why RocksDB and not the alternatives

This is the decision the chapter exists to justify. The node has to persist millions of `hash → bytes` records and look them up by hash, fast, with crash safety, inside one Python process. Three families of alternative were available. Each was a worse fit, for concrete reasons.

Versus MongoDB (a document/NoSQL server). MongoDB is a separate server program: you install it, run it as its own process (often on its own machine), secure it, and your node talks to it over a network socket. For a node whose access pattern is "fetch the value at this hash," every one of those is pure overhead. You pay a network hop (serialize a request, send it, wait, receive, deserialize) for what is, with an embedded store, a local function call. You add a second daemon every operator must deploy, monitor, and keep alive — and a second thing that can be down, misconfigured, or breached. And you gain document features (ad-hoc queries, secondary indexes, aggregation) the node does not use, because it looks vertices up by hash, not by querying their contents. More moving parts, more failure modes, for capabilities the access pattern never exercises.

Versus a SQL database (PostgreSQL, MySQL). The same server objection applies — a separate process and a network hop — but the deeper mismatch is the data model. SQL's strength is the relational model: tables with typed columns, joins across them, declarative `WHERE` queries the engine optimizes. Hathor's storage needs none of that. There is no join to do; the primary access is "value at this key." Mapping a vertex onto columns would mean either decomposing it into a wide, awkward table (and reassembling it on every read) or storing the binary blob in one column — at which point you are using a relational engine as a key-value store and paying for the SQL machinery you never touch. The relational model solves a problem the node does not have, while charging for it in deployment weight and per-row overhead. (SQLite would remove the server objection — it is embedded — but not the model mismatch; you would still be using SQL to emulate `get(key)`.)

Versus plain files (one file per vertex, or a hand-rolled format). This is the tempting "just write bytes to disk myself" option, and it fails on the unglamorous parts a database already solves. You would have to implement: **atomic writes** (a crash mid-write must not leave a torn record — RocksDB's write-ahead log and LSM design give this); **ordered iteration** (the startup index rebuild and the indexes of Chapter 28 need

to scan keys in order — a directory of files does not); **fast lookup at scale** (millions of files in one directory cripples most filesystems); and **compaction** (reclaiming space from superseded metadata without rewriting everything). Each of these is a small database in disguise. RocksDB is that database, written by people who specialize in it.

What it costs, honestly. RocksDB is not free of downsides, and naming them is part of understanding the choice.

- **Key-value only.** There are no ad-hoc queries. You cannot ask the store "which outputs does this address own?" — the data is keyed by hash, and that question is about contents. The answer is the entire reason **indexes** (Chapter 28) exist: they are extra key-value tables, derived from the vertices, that pre-compute the lookups the base store cannot do. The flip side of a key-value store's speed is that every non-hash question must be indexed in advance.
- **A C++ dependency with build friction.** RocksDB is native code, reached through Python bindings. Hathor depends on a **custom git fork** of those bindings (covered in Chapter 13, Poetry) rather than a vanilla PyPI package, which means a heavier, less portable build than pure Python — the price of an embedded native engine.
- **Background compaction.** The LSM design spends CPU and disk I/O compacting in the background. For a write-heavy node this is the right trade, but it is real work happening even when the node looks idle.

The verdict the codebase reaches: the node's access pattern is lookup by hash, write constantly, scan occasionally, survive crashes, deploy as one process. An embedded ordered key-value store is the precise shape of that need, and RocksDB is a mature one. The features the alternatives add — documents, relations, a server — are exactly the features a full node does not use, and every one of them carries a cost it would rather not pay.

27.10 How it plugs into the lifecycle

Storage is among the first things to come alive and among the last to matter, because everything else reads through it.

- **Built in the composition root (Chapter 24).** The `Builder` / `CliBuilder` constructs the `RocksDBStorage` (pointed at `--data <dir>`, or at scratch space for `--temp-data`), then wraps it in a `TransactionRocksDBStorage`, passing in the `VertexParser` (for deserialization), the `IndexesManager`, the reactor, and the settings. Production is always RocksDB; the in-memory storage flag was removed (Chapter 24). The fully-wired storage is handed to the `HathorManager`.

- **Initialized in `manager.start` (Chapter 29).** On start the manager runs the crash check against the storage flag (§27.8), then loads genesis and rebuilds its in-memory view by scanning the store — during which the allow-scope is widened to `ALL` and then narrowed back (§27.7). Any pending on-disk **migrations** run here.
- **Read and written by everything after.** Verification reads parents through the narrow `VertexStorageProtocol`; the vertex handler and consensus write bodies and metadata (`save_transaction`, often `only_metadata=True`); the indexes are populated from the store and consulted by the API; the sync agent fills it as it downloads. The storage layer is the shared ground the rest of the node stands on.

Recap

Concern	Where it lives	Central type / call
Open the DB, hand out column families	<code>hathor/storage/rocksdb_storage.py</code>	<code>RocksDBStorage (:28)</code> , <code>get_or_create_column_family (:93)</code> , <code>create_temp (:73)</code>
Vertex-aware contract (the ABC)	<code>transaction/storage/transaction_storage.py:92</code>	<code>TransactionStorage</code> ; <code>BaseTransactionStorage (:1089)</code>
Production backend	<code>transaction/storage/rocksdb_storage.py:53</code>	<code>TransactionRocksDBStorage</code>
Save a vertex	same	<code>save_transaction (:202) → cache → flush _save_transaction_to_db (:211): bytes(tx) → cf_tx , to_bytes() → cf_meta</code>
Read a vertex	same	<code>_get_transaction (:241, cache→weakref→db) → _get_transaction_from_db (:266) → _load_from_bytes (:170): deserialize + from_bytes</code>
Static metadata (separate)	same	<code>_save_static_metadata (:218)</code> , <code>_load_static_metadata (:222)</code> , <code>cf_static_meta</code>

Concern	Where it lives	Central type / call
Column families (×5)	<code>rocksdb_storage.py:46 – :50</code>	<code>b'tx'</code> , <code>b'meta'</code> , <code>b'static-meta'</code> , <code>b'attr'</code> , <code>b'migrations'</code>
Narrow read interface	<code>vertex_storage_protocol.py:22</code>	<code>VertexStorageProtocol</code>
Walk the DAG	<code>traversal.py</code>	<code>BFSTimestampWalk (:182)</code> , <code>BFSOrderWalk (:218)</code> , <code>DFSWalk (:249)</code>
Gate by validation state	<code>tx_allow_scope.py:25</code>	<code>TxAllowScope</code> , <code>is_allowed (:36)</code> , <code>tx_allow_context (:52)</code>
Crash safety	<code>manager.py</code> + <code>attr</code> family	<code>is_full_node_crashed (manager.py:285)</code> , <code>is_running_manager (:296)</code> , <code>start_running_manager (:343)</code>

The storage layer is where the ledger stops being a graph in memory and becomes bytes on a disk that survive a reboot. Two stacked packages do it: a thin byte store that opens one RocksDB database and slices it into column families, and a vertex-aware store that keys every block and transaction by its hash — the immutable body in one family, the mutable metadata in another, the static metadata in a third — reassembles them on the way back out (through a write-back cache and a weakref identity map), and adds traversal helpers, validation-state gating, and a crash flag riding along. The one thing this layer deliberately cannot do is answer any question that is not "value at this hash." That limitation is the entire premise of the next chapter: **Chapter 28, Indexes** — the derived key-value tables that pre-compute the lookups RocksDB-by-hash can never answer on its own, starting with the one the storage layer cannot: what does this address own?

1. An **abstract base class** (ABC) declares methods without implementing them (marked `@abstractmethod`) so subclasses must supply the bodies. You cannot instantiate an ABC directly; it exists to define a contract. Full treatment in Chapter 1. ↩
2. An **LSM-tree** (log-structured merge-tree) is a storage design that never overwrites data in place. Writes are appended to an in-memory buffer (plus an on-disk log for safety); when the buffer fills it is flushed as a new immutable sorted file; background compaction later merges those files and discards superseded values. It makes writes fast and crash-safe at the cost of occasional extra read work and background CPU. ↩
3. **Compaction** is the LSM-tree's background housekeeping: it merges several sorted on-disk files into fewer, dropping values that have been overwritten or deleted, so reads stay fast and disk space is reclaimed. It runs even when the node is otherwise idle. ↩

4. A **column family** is a named, independent key-space inside a single RocksDB database — like a separate dictionary that shares the same database file and write-ahead log but keeps its own keys and can be tuned and compacted on its own. Hathor uses five: `tx`, `meta`, `static-meta`, `attr`, `migrations`. ↩
5. A **Bloom filter** is a compact, probabilistic membership test: it can answer "definitely not present" with certainty, or "possibly present" (with a small false-positive rate). RocksDB keeps one per data file so a lookup can skip files that certainly do not hold the key. `key_may_exist` exposes this — a fast "no" without touching the data, which is why `transaction_exists` probes it before doing a real read. ↩
6. A **Protocol** (Python `typing.Protocol`) is a structural interface: any object that happens to have the right methods satisfies it, with no explicit inheritance required ("if it walks like a duck..."). It lets a function accept any object shaped correctly, not just one named subclass. Full treatment in Chapter 5. ↩